



طراحی سلسله مراتب حافظه

معماری کامپیوتر – گزارش کار تمرین عملی ششم

استاد

دکتر سربازی آزاد

دانشگاه صنعتی شریف

بهار ۱۴۰۵

محمد مهدی مرادی – ۴۰۳۱۰۶۶۸۱

۳	مقدمه
۳	۱- معماری و ساختار حافظه نهان (L۱ Cache)
۴	۲- ماشین حالت حافظه نهان (FSM)
۶	۳- سیاست نوشتن (Write Policy)
۷	۴- یکپارچه سازی حافظه نهان با پردازنده (Datapath Integration)
۱۱	۵- چالش های تست بنچ، باگ یابی و همگام سازی
۱۲	نتیجه گیری

مقدمه

هدف از این پروژه، ارتقای پردازنده ۳۲ بیتی تک چرخه (طراحی شده در تمرین پنجم) از طریق اضافه کردن حافظه نهان سطح اول (L1 Cache) بود. از آنجا که دسترسی مستقیم پردازنده به حافظه اصلی (Main Memory) زمان‌بر است، حافظه نهان به عنوان یک واسطه پرسرعت بین پردازنده و حافظه اصلی قرار گرفت. در این پروژه، حافظه نهان از پایه طراحی شد، سیاست‌های خواندن و نوشتن در آن پیاده‌سازی گردید و در نهایت به عنوان حافظه دستورات (Instruction Cache) و حافظه داده (Data Cache) در مسیر داده پردازنده (Datapath) قرار گرفت تا برنامه‌های تست نظیر محاسبه دنباله فیبوناچی را با موفقیت اجرا کند.

۱. معماری و ساختار حافظه نهان (L1 Cache)

حافظه نهان طراحی شده دارای ظرفیت ۶۴ بلاک است که هر بلاک آن ۱۲۸ بیت (۱۶ بایت یا ۴ کلمه ۳۲ بیتی) داده را در خود جای می‌دهد. برای مدیریت آدرس‌دهی، آدرس ۳۲ بیتی پردازنده (CPU_Addr) به سه بخش اصلی تقسیم شد:

• **Offset** (بیت‌های ۰ تا ۳): ۴ بیت برای مشخص کردن بایت/کلمه مورد نظر در داخل بلاک ۱۲۸ بیتی.

• **Index** (بیت‌های ۴ تا ۹): ۶ بیت برای آدرس‌دهی ۶۴ بلاک موجود در حافظه نهان.

• **Tag** (بیت‌های ۱۰ تا ۳۱): ۲۲ بیت برای ذخیره شناسه یکتای آدرس و بررسی تطابق (Cache Hit/Miss).

برای نگهداری داده‌ها، از سه آرایه Data_arr (داده‌ها)، tag_arr (تگ‌ها) و valid_arr (بیت‌های اعتبار) استفاده شد. سیگنال cache_hit تنها زمانی یک می‌شود که بیت اعتبار بلاک مورد نظر ۱ باشد و تگ ذخیره شده با تگ آدرس پردازنده برابر باشد.

```
wire [3:0] offset = CPU_Addr[3:0];
wire [5:0] ind = CPU_Addr[9:4];
wire [21:0] tag = CPU_Addr[31:10];

reg [127:0] Data_arr [0:63];
reg [21:0] tag_arr [0:63];
reg valid_arr [0:63];

integer i;
initial begin
    for (i = 0; i < 64; i = i + 1) begin
        valid_arr[i] = 1'b0;
        tag_arr[i] = 22'b0;
        Data_arr[i] = 128'b0;
    end
end

wire cache_hit = (valid_arr[ind] === 1'b1) && (tag_arr[ind] == tag);
wire [127:0] current_data = Data_arr[ind];
```

۲. ماشین حالت حافظه نهان (FSM)

یکی از الزامات کلیدی این بود که درخواست‌های خواندن و نوشتن به حافظه اصلی فقط به مدت ۱ چرخه ساعت (Clock Cycle) ارسال شوند تا از قاطی شدن درخواست‌ها جلوگیری شود. برای پیاده‌سازی دقیق این منطق، یک ماشین حالت ۷ وضعیتی (State-۷ FSM) طراحی شد:

• وضعیت ۰ (IDLE): در صورت رخداد Miss، ماشین حالت بسته به نوع عملیات به وضعیت خواندن یا تخصیص بلاک می‌رود.

• وضعیت ۱ (READ_REQ): سیگنال MM_Read دقیقاً برای ۱ کلاک ۱ می‌شود.

• وضعیت ۲ (READ_WAIT): کش منتظر می‌ماند تا حافظه اصلی داده را آماده کند (MM_Done = ۱).

• وضعیت ۳ (ALLOC_REQ) و ۴ (ALLOC_WAIT): واکنشی کل بلاک از حافظه اصلی برای زمان‌هایی که نوشتن با Miss مواجه شده است.

• وضعیت ۵ (WRITE_REQ) و ۶ (WRITE_WAIT): ارسال پالس ۱ کلاکه برای نوشتن بلاک به روزرسانی شده در حافظه اصلی.

```

localparam IDLE      = 3'd0;
localparam READ_REQ  = 3'd1;
localparam READ_WAIT = 3'd2;
localparam ALLOC_REQ = 3'd3;
localparam ALLOC_WAIT = 3'd4;
localparam WRITE_REQ = 3'd5;
localparam WRITE_WAIT = 3'd6;

reg [2:0] state;

wire [127:0] active_data = (state == READ_WAIT && MM_Done == 1'b1) ? MM_DataIn : current_data;

wire [127:0] updated_block =
    (offset[3:2] == 2'b00) ? {current_data[127:32], CPU_DataIn} :
    (offset[3:2] == 2'b01) ? {current_data[127:64], CPU_DataIn, current_data[31:0]} :
    (offset[3:2] == 2'b10) ? {current_data[127:96], CPU_DataIn, current_data[63:0]} :
    {CPU_DataIn, current_data[95:0]};

```

```

always @(posedge Clk or posedge Rst) begin
    if (Rst === 1'b1) begin
        state <= IDLE;
    end else begin
        case (state)
            IDLE: begin
                if (CPU_Read === 1'b1 && !cache_hit) begin
                    state <= READ_REQ;
                end else if (CPU_Write === 1'b1) begin
                    if (cache_hit) state <= WRITE_REQ;
                    else state <= ALLOC_REQ;
                end
            end
            READ_REQ: begin
                state <= READ_WAIT;
            end
            READ_WAIT: begin
                if (MM_Done === 1'b1) begin
                    valid_arr[ind] <= 1'b1;
                    tag_arr[ind] <= tag;
                    Data_arr[ind] <= MM_DataIn;
                    state <= IDLE;
                end
            end
            ALLOC_REQ: begin
                state <= ALLOC_WAIT;
            end
            ALLOC_WAIT: begin
                if (MM_Done === 1'b1) begin
                    valid_arr[ind] <= 1'b1;
                    tag_arr[ind] <= tag;
                    Data_arr[ind] <= MM_DataIn;
                    state <= WRITE_REQ;
                end
            end
            WRITE_REQ: begin
                state <= WRITE_WAIT;
            end
            WRITE_WAIT: begin
                if (MM_Done === 1'b1) begin
                    Data_arr[ind] <= updated_block;
                    state <= IDLE;
                end
            end
            default: state <= IDLE;
        endcase
    end
end

```

۳. سیاست نوشتن (Write Policy)

با توجه به اینکه پردازنده در هر دستور ۳۲ بیت داده تولید می‌کند اما عرض گذرگاه حافظه اصلی ۱۲۸ بیت است، سیاست **Write-Allocate** پیاده‌سازی شد. روند کار به این صورت است: اگر پردازنده بخواهد روی آدرسی بنویسد که در کش نیست (Write Miss)، کش ابتدا کل بلاک ۱۲۸ بیتی مربوطه را از حافظه اصلی می‌خواند (Allocation). سپس کلمه ۳۲ بیتی جدید را در جایگاه مناسب (با استفاده از offset) با سایر کلمات ترکیب کرده و در نهایت کل بلاک ۱۲۸ بیتی آپدیت‌شده را به حافظه اصلی بازمی‌گرداند.

```
assign MM_Addr = {CPU_Addr[31:4], 4'b0000};
assign MM_DataOut = updated_block;

assign MM_Read = (state == READ_REQ) || (state == ALLOC_REQ);
assign MM_Write = (state == WRITE_REQ);

assign CPU_Done = (state == IDLE && cache_hit && !CPU_Write) ||
                 (state == READ_WAIT && MM_Done == 1'b1) ||
                 (state == WRITE_WAIT && MM_Done == 1'b1);

wire [31:0] word_0 = active_data[31:0];
wire [31:0] word_1 = active_data[63:32];
wire [31:0] word_2 = active_data[95:64];
wire [31:0] word_3 = active_data[127:96];

assign CPU_DataOut = (offset[3:2] == 2'b00) ? word_0 :
                    (offset[3:2] == 2'b01) ? word_1 :
                    (offset[3:2] == 2'b10) ? word_2 :
                    word_3;
```

۴. یکپارچه‌سازی حافظه نهان با پردازنده (Datapath Integration)

اضافه کردن حافظه نهان به پردازنده تک‌چرخه، صرفاً یک اتصال ساده پورت‌ها نبود؛ بلکه نیازمند تغییرات اساسی در منطق کنترل (Control Logic) و توقف پردازنده (Freezing/Stalling) بود تا معماری هاروارد (Harvard Architecture) به درستی پیاده‌سازی شود و پردازنده در زمان تاخیرهای حافظه (Miss Penalty) دچار اختلال نگردد. در این راستا، اقدامات کلیدی زیر در ماژول Main انجام شد:

• تفکیک حافظه دستور و داده (L1i و L1d): دو نمونه (Instance) کاملاً مستقل از ماژول L1_Cache ساخته شد.

نمونه اول (inst_cache) به صورت فقط‌خواندنی (CPU_Write = ۰ و CPU_Read = ۱) به صورت دائم به شمارنده برنامه (PC) متصل شد تا دستورات را واکنشی کند. نمونه دوم (data_cache) به خروجی ALU و پورت‌های داده متصل گردید. این تفکیک فیزیکی اجازه می‌دهد تا در یک چرخه کاری پردازنده، هیچ تداخلی بین واکنشی دستور و دسترسی به داده‌ها پیش نیاید.


```

//PC
reg [31:0] PC_reg = 32'd2044;
wire [31:0] next_pc;
wire [31:0] pc_plus_4 = PC_reg + 32'd4;
wire alu_ready;

assign PC = PC_reg >> 2;

always @(posedge Clk or posedge Rst) begin
    if (Rst)
        PC_reg <= 32'b0;
    else if (alu_ready & cpu_ready)
        PC_reg <= next_pc;
end
//

wire inst_mm_write_nc;
wire [127:0] inst_mm_dataout_nc;

L1_Cache inst_cache (
    .Clk(Clk),
    .Rst(Rst),
    .CPU_Write(1'b0),
    .CPU_Read(1'b1),
    .MM_Done(InstrMM_Done),
    .CPU_DataIn(32'b0),
    .CPU_Addr(PC_reg),
    .MM_DataIn(InstrMM_DataIn),
    .CPU_Done(inst_cache_done),
    .MM_Write(inst_mm_write_nc),
    .MM_Read(InstrMM_Read),
    .CPU_DataOut(InstrIn),
    .MM_Addr(InstrMM_Addr),
    .MM_DataOut(inst_mm_dataout_nc)
);

```

```

//ALU
wire [31:0] alu_in2;
wire [31:0] alu_result;
wire alu_zero;

assign alu_in2 = ALUSrc ? sign_ext_imm : read_data2;

ALU alu_inst (
    .clk(Clk),
    .ALUOp(ALUOp),
    .Reg1(read_data1),
    .Reg2(alu_in2),
    .Result(alu_result),
    .zero(alu_zero),
    .ready(alu_ready)
);
//

wire valid_MemRead = MemRead & inst_cache_done;
wire valid_MemWrite = MemWrite & inst_cache_done;
wire is_memory_instruction = valid_MemRead | valid_MemWrite;
wire [31:0] safe_data_addr = is_memory_instruction ? alu_result : 32'h00004000;

L1_Cache data_cache (
    .Clk(Clk),
    .Rst(Rst),
    .CPU_Write(valid_MemWrite),
    .CPU_Read(valid_MemRead),
    .MM_Done(DataMM_Done),
    .CPU_DataIn(read_data2),
    .CPU_Addr(safe_data_addr),
    .MM_DataIn(DataMM_DataIn),
    .CPU_Done(data_cache_done),
    .MM_Write(DataMM_Write),
    .MM_Read(DataMM_Read),
    .CPU_DataOut(DataIn),
    .MM_Addr(DataMM_Addr),
    .MM_DataOut(DataMM_DataOut)
);

```

• **طراحی منطق توقف جامع (Master Stall Logic):** از آنجا که دسترسی به کش ممکن است با Miss مواجه شود و چند کلاک به طول بینجامد، سیگنال `cpu_ready` به عنوان یک ترمز یکپارچه برای کل پردازنده طراحی شد. منطق این سیگنال با استفاده از عبارت زیر پیاده‌سازی گردید: $cpu_ready = inst_cache_done \&\& (valid_MemRead \mid \mid valid_MemWrite) \mid \mid data_cache_done$ این منطق تضمین می‌کند که پردازنده تنها زمانی به کلاک بعدی می‌رود که: اولاً دستور فعلی از کش دستورات با موفقیت خوانده شده باشد، و ثانیاً اگر دستور فعلی نیاز به کار با حافظه داده دارد (مثل دستورات LW یا SW)، عملیات کش داده نیز با موفقیت خاتمه یافته باشد.

• **محافظت از حافظه داده و آدرس‌دهی ایمن (Safe Memory Access):** یکی از چالش‌های بزرگ در اتصال پردازنده به تست‌بنچ، ارسال درخواست‌های کاذب (False Requests) به حافظه داده، پیش از آماده شدن و دیکود شدن دستور بود. برای حل این مشکل دو لایه امنیتی ایجاد شد: ۱. **فیلتر سیگنال‌های کنترلی:** سیگنال‌های خواندن و نوشتن (`MemRead` و `MemWrite`) با سیگنال `inst_cache_done` ترکیب (`AND`) شدند تا سیگنال‌های معتبری به نام `valid_MemRead` و `valid_MemWrite` ساخته شوند. این یعنی تا زمانی که دستور به طور کامل از حافظه واکنشی و رمزگشایی نشده است، پردازنده حق ارسال هیچ درخواستی به کش داده را ندارد. ۲. **مدیریت آدرس‌های بی‌اثر:** برای جلوگیری از خطاهای Out-of-Bounds (آدرس‌های خارج از محدوده) در محیط شبیه‌ساز، یک آدرس امن (`safe_data_addr`) طراحی شد. اگر دستور فعلی از نوع دسترسی به حافظه نباشد، به جای ارسال خروجی تصادفی ALU به پورت آدرس کش، یک آدرس امن و خنثی (مانند `h'00004000'32`) ارسال می‌شود تا سیستم دچار رفتار پیش‌بینی نشده نگردد.

```

wire valid_MemRead = MemRead & inst_cache_done;
wire valid_MemWrite = MemWrite & inst_cache_done;
wire is_memory_instruction = valid_MemRead | valid_MemWrite;
wire [31:0] safe_data_addr = is_memory_instruction ? alu_result : 32'h00004000;

```

• **توقف (Freezing) ثبات‌ها و PC:** برای جلوگیری از اجرای ناقص دستورات و از بین رفتن داده‌های پردازنده در کلاک‌هایی که حافظه نهان یا ALU (در عملیات ضرب و تقسیم) درگیر هستند، سیگنال به‌روزرسانی ثبات‌ها (`RegWrite`) و کلاک PC با سیگنال‌های (`alu_ready & cpu_ready`) ترکیب شدند. در نتیجه، در طول زمان‌هایی که پردازنده منتظر پاسخ حافظه است، وضعیت تمامی رجیسترها کاملاً فریز شده و هیچ تغییر مخربی (`Corrupted State`) در سیستم ثبت نمی‌شود.

```

always @(posedge Clk or posedge Rst) begin
    if (Rst)
        PC_reg <= 32'b0;
    else if (alu_ready & cpu_ready)
        PC_reg <= next_pc;
end

```

```

RF register_file (
    .Clk(Clk),
    .Rst(Rst),
    .RegWrite(RegWrite & alu_ready & cpu_ready),
    .ReadReg1(rs),
    .ReadReg2(rt),
    .WriteReg(write_reg),
    .Data(write_data),
    .ReadData1(read_data1),
    .ReadData2(read_data2),
    .R0(R0), .R1(R1), .R2(R2), .R3(R3), .R4(R4), .R5(R5), .R6(R6), .R7(R7),
    .R8(R8), .R9(R9), .R10(R10), .R11(R11), .R12(R12), .R13(R13), .R14(R14), .R15(R15),
    .R16(R16), .R17(R17), .R18(R18), .R19(R19), .R20(R20), .R21(R21), .R22(R22), .R23(R23),
    .R24(R24), .R25(R25), .R26(R26), .R27(R27), .R28(R28), .R29(R29), .R30(R30), .R31(R31)
);
//

```

۵. چالش‌های تست‌بنچ، باگیابی و همگام‌سازی

طی پیاده‌سازی و تست این مدار با تست‌بنچ‌های داوری، چالش‌های مختلفی بروز کرد که با تکنیک‌های مهندسی برطرف شدند:

- **رفع خطای Multiple Drivers (حالت نامعلوم X):** در ابتدا، آرایه‌های کش هم در بلوک initial و هم در زمان Rst مقداری می‌شدند. شبیه‌ساز (iverilog) این را به عنوان تداخل درایورها شناسایی می‌کرد که باعث می‌شد مقدار valid_arr تبدیل به X (نامعلوم) شده و تست‌بنچ در حالت not ready! قفل کند (Deadlock). این مشکل با حذف حلقه ریست از بلوک always و بسنده کردن به بلوک initial کاملاً حل شد.

- **همگام‌سازی PC (خروجی mm_pc):** برای رفع مشکل ردیابی دستورات توسط تست‌بنچ اصلی پردازنده، ما نیاز به مطلع بودن از شماره دستور (Instruction Index) داشتیم. یک پورت خروجی جدید به ماژول پردازنده اضافه شد که مقدار $PC_reg \gg 2$ (تقسیم آدرس بر ۴) را مستقیماً به داور گزارش می‌دهد تا ipc (Instruction PC) با ماشین وضعیت شبیه‌ساز هماهنگ بماند.

نتیجه گیری

طراحی یک ماشین حالت اصولی برای کش و رعایت زمان بندی دقیق ۱ کلاکه در درخواست های حافظه، باعث شد تا پردازنده بتواند توقف های لازم (Stalls) را به درستی مدیریت کند. با رفع تداخل های شبیه ساز و همگام سازی سیگنال های کنترلی با استفاده از انتساب های $\Rightarrow$ ، پردازنده موفق شد برنامه های پیچیده ای چون تبدیل مبنا و محاسبه فیبوناچی را بر روی بستر حافظه نهان بدون خطا اجرا کرده و تاییدیه نهایی (ACCEPTED) را دریافت نماید. در آخر هم در شکل زیر خروجی گرفته شده از ران کردن تست بنچ ها در ترمینال را مشاهده میکنید.

```
mahdi@mahdi-ASUS-Zenbook-14:~/Documents/CA/HWs/Practicals/ca4042-judge$ ./validate-verilog.sh ../6/HW6-Q1-403106681.v HW6/cache-tb.v
ACCEPTED
      500 /      500
HW6/cache-tb.v:177: $finish called at 161681 (1s)
mahdi@mahdi-ASUS-Zenbook-14:~/Documents/CA/HWs/Practicals/ca4042-judge$ ./validate-verilog.sh ../6/HW6-Q2-403106681.v HW6/cpu-tb.v
fib(10) = exp: 55 real: 55
fib(11) = exp: 89 real: 89
product = exp: 4895 real: 4895
base3 digits LSD first:
2 2 0 1 0 2 0 2
base3 = 20201022
ACCEPTED
      208 /      208
HW6/cpu-tb.v:371: $finish called at 27014 (1s)
mahdi@mahdi-ASUS-Zenbook-14:~/Documents/CA/HWs/Practicals/ca4042-judge$ |
```